МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский авиационный институт (национальный исследовательский университет)»

Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Отчёт по лабораторной работе № 1

по курсу «Математическое моделирование»

на тему «Определение параметров за косым скачком уплотнения на клине с использованием численных методов решения нелинейных уравнений»

Вариант 9

Выполнил: Жуховицкий А. Д.

Группа: М8О-303Б-23

Преподаватель: _____

Оценка: _____

Дата: _____

Москва, 2026

СОДЕРЖАНИЕ

1 Теория.....3
1.1 Отделение корней.....4
1.2 Метод Ньютона (метод касательных).....5
1.3 Метод секущих.....7
1.4 Задача определения угла присоединённого скачка уплотнения на клине.....8
2 Исходные данные.....12
3 Основная часть.....13
3.1 Задание 1.....13
3.2 Задание 2.....16
3.3 Задание 3.....19
4 Вывод.....22
Список использованных источников.....23
Приложение А. Программный код.....24

1 Теория

Цель работы — изучение условно и безусловно сходящихся итерационных методов решения нелинейных уравнений и их применение к задаче определения параметров потока за косым скачком уплотнения (КСУ) на клине.

Одной из наиболее частых задач, с которыми сталкиваются при вычислительных экспериментах, является решение нелинейных уравнений вида

$$f(x)=0, \tag{1}$$

где $f(x)$ — функция, определённая и непрерывная на некотором интервале $a<x<b$. В некоторых случаях на функцию $f(x)$ накладываются дополнительные ограничения, например непрерывность первой и второй производных.

Нелинейные уравнения подразделяются на алгебраические и трансцендентные. Алгебраические уравнения содержат только алгебраические функции (рациональные, иррациональные, целые) и могут быть представлены в виде полинома n-й степени с действительными коэффициентами. Трансцендентные уравнения содержат показательные, тригонометрические, логарифмические и другие функции. Для достаточно сложных уравнений не всегда можно найти точное решение, поэтому применяют приближённые (численные) методы — методы последовательных приближений, или итерационные методы. Начальное приближение может находиться из физических соображений, из опыта решения аналогичных задач, с помощью графических методов и т. д.

Условие существования корня уравнения (1) следует из следствия теоремы Больцано–Коши: *если непрерывная функция $f(x)$ принимает значения разных знаков на концах отрезка $[a,b]$, т. е. $f(a)\cdot f(b)<0$, то внутри этого отрезка содержится по крайней мере один корень уравнения $f(x)=0$. Если же $f(x)$ непрерывна, дифференцируема и её первая производная*

сохраняет знак внутри отрезка $[a,b]$*, то на данном отрезке находится только один корень.*

Таким образом, при нахождении корней уравнения (1) численным методом, кроме непрерывности $f(x)$, предполагается, что:

- функция принимает на концах отрезка разные знаки;
- производные непрерывны на отрезке;
- производные на отрезке не меняют знака.

Геометрически последнее условие означает, что реализуется одна из четырёх схем, приведённых на рисунке 1.1.

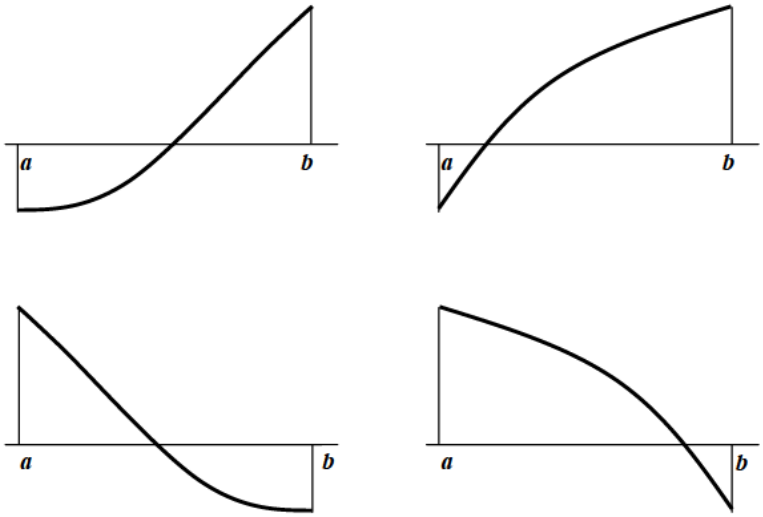


Рисунок 1.1 – Геометрическая трактовка знакопостоянства производных

1.1 Отделение корней

Приближённое нахождение изолированных действительных корней уравнения (1) осуществляется в два этапа:

1. Находятся отрезки $[a_i,b_i]$, внутри каждого из которых содержится один и только один корень уравнения. Этот этап называется процедурой отделения корней; по сути, на нём осуществляется грубое нахождение корней $x=x_i^{\cdot}$.
2. Грубое значение каждого корня $x=x_i^{\cdot}$ уточняется до заданной точности одним из численных методов, в которых реализуются последовательные приближения.

Отделение корней начинается с установления знаков $f(x)$ в граничных точках области определения функции. Затем аналитически или графически находят значения функции в некоторых промежуточных точках $x=x_1, x_2, \ldots$ и выбирают интервалы, на концах которых функция имеет разные знаки. После этого необходимо убедиться, что в каждом интервале находится только один корень, в противном случае интервал изменяют.

Если известны корни уравнения $f'(x)=0$, процесс отделения корней можно упростить: достаточно определить знаки функции $f(x)$ в точках нулей её производной и в граничных точках $x=a$ и $x=b$. Действительные корни уравнения можно отделить и графически — как точки пересечения графика $y=f(x)$ с осью абсцисс. Достоинством графического метода, кроме наглядности, является то, что он часто даёт возможность оценить количество корней и их знаки.

1.2 Метод Ньютона (метод касательных)

Метод Ньютона относится к методам, в основе которых лежит замена исследуемой функции $f(x)$ более простой функцией, в данном случае касательной. Геометрически в начальной точке x_0 проводится касательная к кривой $f(x)$ и находится её пересечение с осью абсцисс (рисунок 1.2).

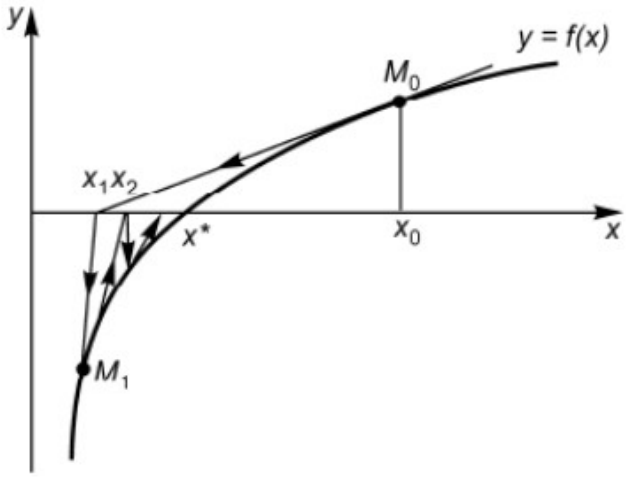


Рисунок 1.2 – Метод Ньютона

Уравнение касательной к кривой $f(x)$ в точке $M_0(x_0, f(x_0))$ имеет вид

$$y - f(x_0) = f'(x_0)(x - x_0),$$

а следующее приближение x_1, являющееся точкой пересечения касательной с осью абсцисс, даётся формулой

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

Аналогичным образом, строя касательные последовательно из точек $M_1, \ldots, M_{n-1}$, находятся следующие приближения:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, f'(x_n) \neq 0. \quad (2)$$

Метод касательных является **условно сходящимся** методом: для его сходимости ($x_i \to x^*$ при $i \to \infty$) в области поиска корня должно выполняться условие

$$|f(x)f''(x)| < (f'(x))^2. \quad (3)$$

Выполнения условий следствия теоремы Больцано–Коши недостаточно для того, чтобы метод сходился. Проверка условия (3) обязательна на **всём** промежутке поиска корня; если она не выполняется, сходимость будет лишь в некоторой окрестности корня. Производные в методе Ньютона находятся аналитически; любое изменение подхода к вычислению производных переводит метод в другой, например в метод секущих.

Для окончания итерационного процесса могут быть использованы следующие критерии:

1. Максимальное число итераций — критерий необходим на случай, если метод не сходится.
2. Слабая вариация приближения к корню: $|x_{n+1} - x_n| < \varepsilon$ или $|x_{n+1} - x_n| < \varepsilon |x_n|$.
3. Достаточно малое значение функции: $|f(x_{n+1})| < \varepsilon_1$.

Метод Ньютона обладает сходимостью второго порядка: вблизи корня погрешность уменьшается по закону

$$\varepsilon_i = const \cdot \varepsilon_{i-1}^2. \quad (4)$$

Здесь ε_i — ошибка относительно неизвестного заранее точного решения, поэтому формулу (4) нельзя использовать в качестве критерия

сходимости. Однако ею можно воспользоваться для сравнения по итерациям после получения корня с заданной точностью, принимая его за точный. Итерации по методу Ньютона сходятся очень быстро; невыполнение критерия окончания при $i>10\div15$ обычно указывает на отсутствие сходимости, ошибку в формулах или в программе.

Метод касательных имеет достаточно жёсткие ограничения: он не может применяться при нарушении знакопостоянства производных, при существовании неограниченных вторых производных и т. п. Если условие знакопостоянства нарушено даже вдали от корня, где выбрано x_0, а вблизи корня выполняется, метод всё равно неприменим без сужения начального отрезка (рисунок 1.3).

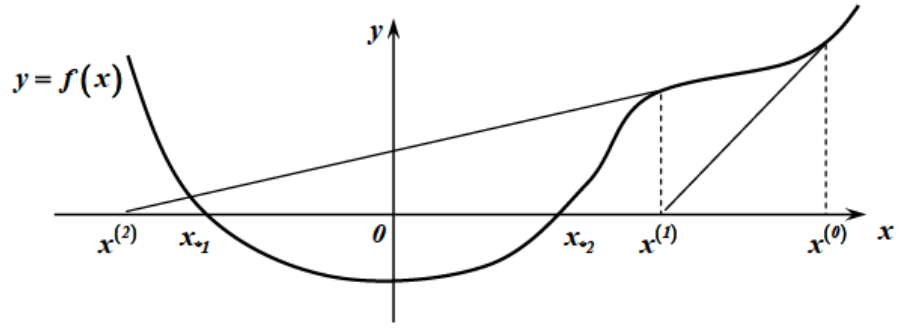


Рисунок 1.3 – Неприменимость метода Ньютона

1.3 Метод секущих

Вычисление производной $f'(x)$, необходимой в методе Ньютона, не всегда бывает удобно или возможно. Замена производной первой разделённой разностью, которую находят по двум последним итерациям (т. е. замена касательной на секущую), приводит к методу секущих. В качестве аппроксимирующей берётся прямая, проходящая через две последние точки x_n и x_{n-1}, т. е. вместо производной в методе касательных подставляется

$$f'(x_n)\approx\frac{f(x_n)-f(x_{n-1})}{x_n-x_{n-1}}.$$

Тогда приходим к формуле метода секущих:

$$x_{n+1}=x_n-\frac{x_n-x_{n-1}}{f(x_n)-f(x_{n-1})}f(x_n). \qquad (5)$$

Метод секущих является двухшаговым, т. е. требует двух начальных (разгонных) точек x_0 и x_1. Он позволяет брать за x_0 и x_1 граничные значения начального отрезка, если на нём выполняются условия следствия теоремы Больцано–Коши. Графически метод иллюстрируется на рисунке 1.4.

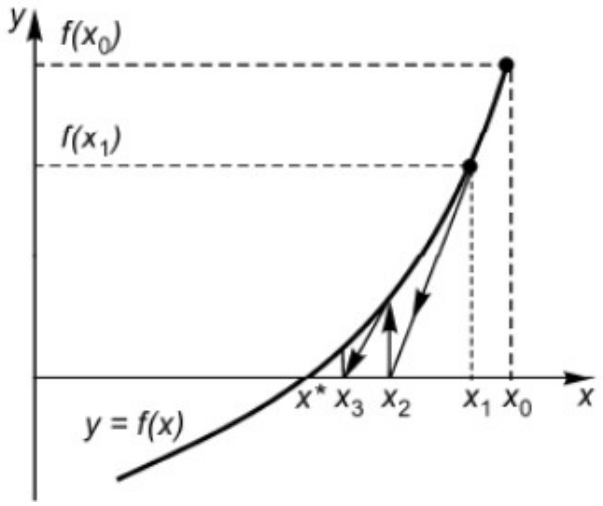


Рисунок 1.4 – Метод секущих

Сначала через точки $(x_0, f(x_0))$ и $(x_1, f(x_1))$ проводится прямая и определяется x_2, а вертикальная прямая в точке x_2 даёт $f(x_2)$. Далее прямая проводится через точки $(x_1, f(x_1))$ и $(x_2, f(x_2))$ и т. д., пока не будет выполнено одно из условий окончания итерационного процесса $|x_{n+1} - x_n| < \varepsilon$ или $|f(x_n)| < \varepsilon_1$.

Обычно в методе секущих требуется больше итераций, чем в методе касательных, но каждая итерация выполняется быстрее, так как не требуется вычислять $f'(x)$. Поэтому при том же объёме вычислений часто можно сделать больше итераций и получить более высокую точность.

1.4 Задача определения угла присоединённого скачка уплотнения на клине

Главная особенность сверхзвуковых потоков состоит в том, что в них при условиях торможения образуются поверхности разрыва, на которых происходит скачкообразное изменение параметров потока. Такие поверхности, положение которых не изменяется во времени, называют скачками уплотнения (СУ). Наиболее распространённый случай их

формирования соответствует обтеканию тел, движущихся прямолинейно с постоянной сверхзвуковой скоростью (рисунок 1.5).

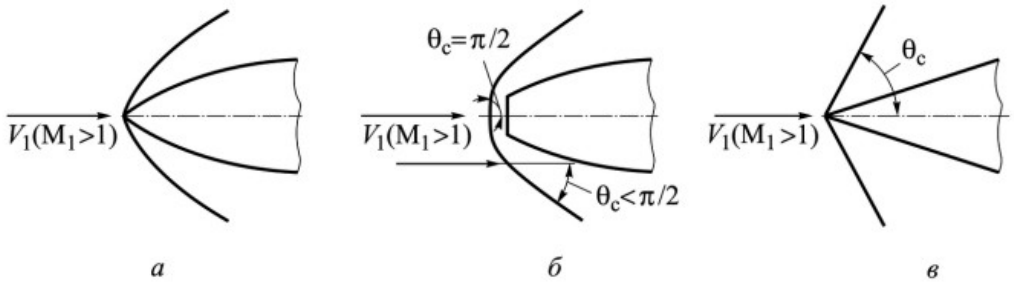


Рисунок 1.5 – Разновидности скачков уплотнения

Присоединённый криволинейный СУ (рисунок 1.5, а) образуется при обтекании заострённого тела вращения криволинейной формы, отсоединённый криволинейный СУ (рисунок 1.5, б) — перед затупленной частью поверхности тела. При обтекании заострённой части тела с прямолинейной образующей возникает прямолинейный присоединённый скачок уплотнения (рисунок 1.5, в). Если поверхность скачка ориентирована нормально к вектору скорости v_1 набегающего потока, скачок называется прямым (ПСУ), если наклонена под некоторым углом — косым (КСУ).

Конечной целью является получение параметров за ударной волной, а также угла наклона скачка на клине (рисунок 1.6).

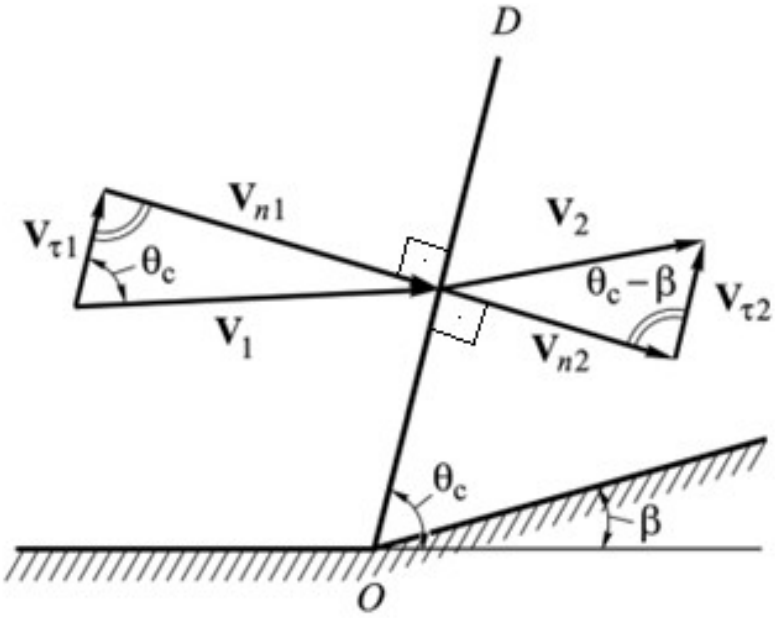


Рисунок 1.6 – Схема разложения скоростей до и после КСУ

Из рисунка 1.6 определяются нормальные (индекс «n») к КСУ и касательные (индекс «τ») составляющие скоростей потока:

$$v_{1n}=v_1\sin\theta_c, v_{1\tau}=v_1\cos\theta_c, v_{2n}=v_2\sin(\theta_c-\beta), v_{2\tau}=v_2\cos(\theta_c-\beta). \quad (6)$$

Полагая, что касательные составляющие скорости до и после скачка равны, получаем зависимость для определения наклона скачка уплотнения:

$$\frac{v_{1n}}{v_{2n}}=\frac{tg\theta_c}{tg(\theta_c-\beta)}. \quad (7)$$

Воздух принимается совершенным невязким газом с уравнением состояния $p=\rho RT$, где p — статическое давление, Па; ρ — плотность, кг/м³; $R=287$ Дж/(кг·К) — газовая постоянная воздуха; T — статическая температура, К. Скорость звука определяется как $a=\sqrt{\kappa RT}$, где $\kappa=1,4$ — показатель адиабаты воздуха.

Для определения угла наклона скачка используется уравнение

$$f(\theta_c)=\sin^2\theta_c-\frac{\kappa+1}{2}\cdot\frac{\sin\theta_c\sin\beta}{\cos(\theta_c-\beta)}-\left(\frac{1}{M_1}\right)^2=0, \quad (8)$$

где β — угол клина; θ_c — угол наклона скачка уплотнения; $M_1=v_1/a_1$ — число Маха набегающего потока.

При решении уравнения (8) на промежутке $\theta_c\in(0,\pi/2)$ больший корень отбрасывается, поскольку он соответствует неустойчивому стационарному решению: поток сжимается с большим приращением энтропии на КСУ (теорема Пригожина). Такое решение сильно подвержено малейшим возмущениям давления в среде, которые могут создать нестационарную ударную волну либо перевести систему в устойчивое стационарное состояние с меньшим углом КСУ.

Плотность и давление за косым скачком уплотнения рассчитываются по формулам

$$\rho_2=\frac{\rho_1}{1-u_1}, p_2=p_1+\rho_1u_1v_{1n}^2, \quad (9)$$

где u_1 — безразмерный параметр:

$$u_1 = 1 - \frac{\sin(\theta_c - \beta)\cos\theta_c}{\sin\theta_c \cos(\theta_c - \beta)}. \tag{10}$$

2 Исходные данные

Исходные данные варианта 9 для задания 1 приведены в таблице 2.1, для задания 2 — в таблице 2.2.

Таблица 2.1 – Исходные данные для задания 1 ($\kappa = 1{,}4$, $R = 287$ Дж/(кг·К))

Вариант	p_1, **Па**	T_1, **К**	M_1	β, °	**Метод**
9	75 000	288	3	10	МС

Таблица 2.2 – Исходные данные для задания 2

Вариант	**Уравнение**	$[a, b]$	**Метод**
9	$x^4 - 3x^3 + 5x - 10 = 0$	$[-5; 0]$	МН

Обозначения: МС — метод секущих, МН — метод Ньютона. В задании 3 используются исходные данные из таблицы 2.1 и метод из таблицы 2.2, т. е. уравнение (8) решается методом Ньютона.

Во всех расчётах итерации прекращаются по критерию слабой вариации приближения $|x_{n+1} - x_n| < \varepsilon$ с $\varepsilon = 10^{-6}$; максимальное число итераций — 100. Программа написана на Python 3.14 в Jupyter Notebook с использованием библиотек NumPy, pandas и Matplotlib. Численные методы реализованы самостоятельно; библиотека SciPy применяется только для проверки полученных корней. Полный код приведён в приложении А.

3 Основная часть

3.1 Задание 1

Требуется определить угол наклона КСУ θ_c из уравнения (8) на интервале $(0,\pi/2)$ методом секущих, рассчитать параметры p_2, T_2, ρ_2, M_2 за КСУ и определить режим течения после скачка.

Отделение корней. При подстановке исходных данных $\frac{\kappa+1}{2}\sin\beta = 1{,}2\sin 10° = 0{,}208$ и $(1/M_1)^2 = 1/9 = 0{,}111$ уравнение (8) принимает вид

$$f(\theta_c) = \sin^2\theta_c - 0{,}208\frac{\sin\theta_c}{\cos(\theta_c - 10°)} - 0{,}111 = 0. \qquad (11)$$

График функции на интервале $(0°, 90°)$ приведён на рисунке 3.1. Табулирование функции показывает, что уравнение имеет два корня: $\theta_c \approx 27{,}4°$ (слабый скачок) и $\theta_c \approx 86{,}4°$ (сильный скачок). Больший корень отбрасывается (см. п. 1.4).

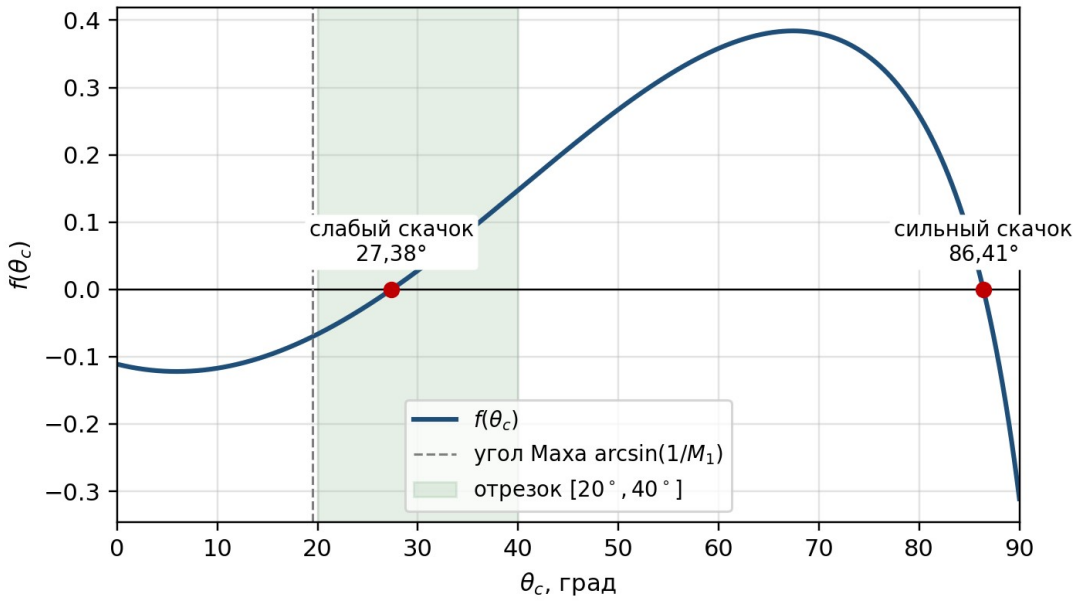


Рисунок 3.1 – График функции $f(\theta_c)$ на интервале $(0,\pi/2)$

Для уточнения меньшего корня выбран отрезок $[a,b] = [20°, 40°] = [0{,}349; 0{,}698]$ рад. Левая граница лежит правее угла Маха $\arcsin(1/M_1) = 19{,}5°$, меньше которого угол скачка быть не может; правая

граница лежит заведомо левее точки максимума функции ($\approx 67{,}4°$, где f' меняет знак) и второго корня.

Аналитические производные. Обозначим $g(\theta)=\dfrac{\sin\theta}{\cos(\theta-\beta)}$. Учитывая, что

$$g'(\theta)=\frac{\cos\theta\cos(\theta-\beta)+\sin\theta\sin(\theta-\beta)}{\cos^2(\theta-\beta)}=\frac{\cos\beta}{\cos^2(\theta-\beta)},\quad g''(\theta)=\frac{2\cos\beta\sin(\theta-\beta)}{\cos^3(\theta-\beta)},$$

получаем первую и вторую производные функции (8):

$$f'(\theta_c)=\sin 2\theta_c-\frac{\kappa+1}{2}\cdot\frac{\sin\beta\cos\beta}{\cos^2(\theta_c-\beta)}, \tag{12}$$

$$f''(\theta_c)=2\cos 2\theta_c-(\kappa+1)\frac{\sin\beta\cos\beta\sin(\theta_c-\beta)}{\cos^3(\theta_c-\beta)}. \tag{13}$$

Метод секущих производных не требует; формулы (12)–(13) используются для проверки условий на отрезке, а также в методе Ньютона в задании 3.

Проверка существования единственного корня. Проверки реализованы функцией `check_root_conditions` (приложение А): отрезок разбивается на 10 000 точек, в каждой вычисляются f' и f''.

1. $f(20°)=-0{,}0665<0$, $f(40°)=0{,}147>0$, т. е. $f(a)\cdot f(b)<0$ — корень на отрезке существует.
2. $0{,}431\le f'(\theta_c)\le 0{,}711$, т. е. $f'>0$ на всём отрезке — функция строго возрастает, корень единственный.
3. $0{,}0314\le f''(\theta_c)\le 1{,}46$, т. е. $f''>0$ — вторая производная также сохраняет знак (точка перегиба $\approx 40{,}4°$ лежит вне отрезка).

Все условия следствия теоремы Больцано–Коши выполнены, поэтому в качестве разгонных точек метода секущих взяты границы отрезка: $x_0=20°$, $x_1=40°$. Функция и её производные на отрезке показаны на рисунке 3.2.

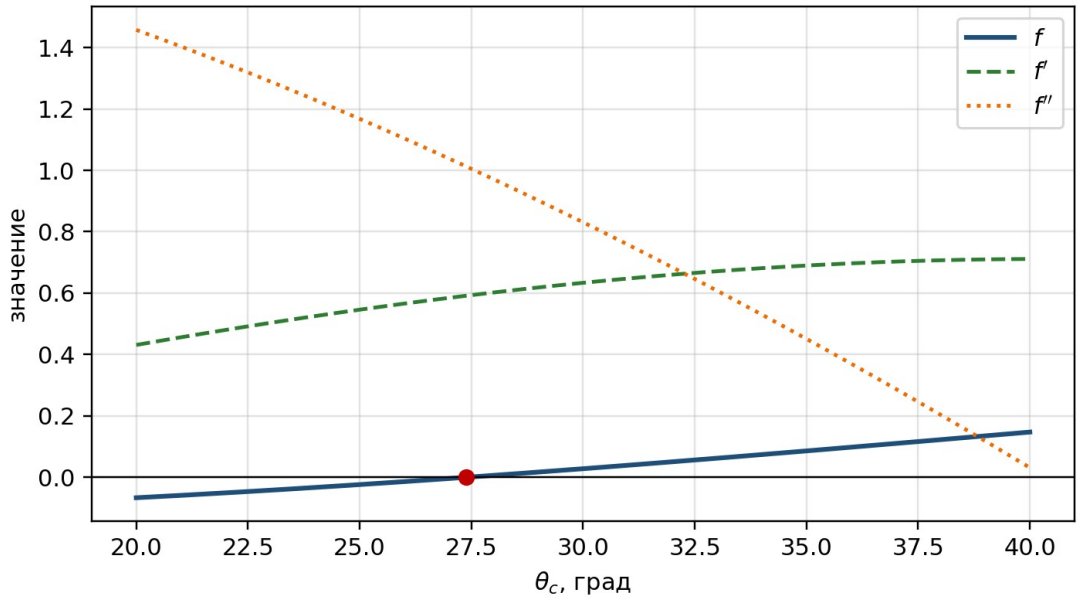


Рисунок 3.2 – Функция $f(\theta_c)$ и её производные на отрезке $[20°, 40°]$

Решение методом секущих. Итерации по формуле (5) выполнялись до выполнения условия $|x_{n+1} - x_n| < \varepsilon$. Последовательность приближений приведена в таблице 3.1; для анализа сходимости найденный корень принят за точный и вычислена ошибка $\varepsilon_n = |\theta_n - \theta^{*}|$.

Таблица 3.1 – Итерации метода секущих в задании 1

n	θ_n, °	$f(\theta_n)$	$\varepsilon_n = \lvert\theta_n - \theta^{*}\rvert$, **рад**
0	20,0	−0,0665	0,129
1	40,0	0,147	0,220
2	26,2	−0,0118	0,0203
3	27,2	−0,00147	0,00249
4	27,4	$2{,}69 \cdot 10^{-5}$	$4{,}54 \cdot 10^{-5}$
5	27,4	$-5{,}75 \cdot 10^{-8}$	$9{,}72 \cdot 10^{-8}$
6	27,4	$-2{,}24 \cdot 10^{-12}$	$3{,}78 \cdot 10^{-12}$

Корень найден за 5 итераций: $\theta_c = 0{,}478$ рад, или $\theta_c = 27{,}4°$.

Параметры за скачком. Параметры набегающего потока:

$$\rho_1 = \frac{p_1}{R T_1}, \; a_1 = \sqrt{\kappa R T_1}, \; v_1 = M_1 a_1. \tag{14}$$

После определения θ_c по формулам (6), (9), (10) находятся v_{1n}, u_1, ρ_2 и p_2, а температура за скачком — из уравнения состояния $T_2 = p_2/(\rho_2 R)$. Касательная составляющая скорости на скачке сохраняется: $v_{2\tau} = v_{1\tau}$.

Нормальная составляющая находится из уравнения неразрывности $\rho_1 v_{1n} = \rho_2 v_{2n}$, откуда с учётом (9) $v_{2n} = v_{1n}(1 - u_1)$. Тогда

$$v_2 = \sqrt{v_{2n}^2 + v_{2\tau}^2},\quad a_2 = \sqrt{\kappa R T_2},\quad M_2 = \frac{v_2}{a_2}. \tag{15}$$

Расчёт выполнен функцией `shock_parameters` (приложение А), результаты сведены в таблицу 3.2.

Таблица 3.2 – Результаты решения задания 1

Параметр	**Обозначение**	**Значение**	**Размерность**
Угол наклона КСУ	θ_c	$27{,}4$	°
Плотность набегающего потока	ρ_1	$0{,}907$	кг/м³
Скорость звука до скачка	a_1	340	м/с
Скорость набегающего потока	v_1	1020	м/с
Безразмерный параметр	u_1	$0{,}396$	—
Давление за КСУ	p_2	$1{,}54 \cdot 10^5$	Па
Температура за КСУ	T_2	358	К
Плотность за КСУ	ρ_2	$1{,}50$	кг/м³
Скорость за КСУ	v_2	950	м/с
Скорость звука за КСУ	a_2	379	м/с
Число Маха за КСУ	M_2	$2{,}51$	—
Режим течения после КСУ	—	сверхзвуковой ($M_2 > 1$)	—

Так как $M_2 = 2{,}51 > 1$, **течение после КСУ остаётся сверхзвуковым**. За скачком давление возрастает в 2,05 раза, температура — в 1,24 раза, плотность — в 1,65 раза; скорость потока уменьшается, а скорость звука растёт, поэтому число Маха падает с 3 до 2,51.

Для контроля параметры за скачком сопоставлены с соотношениями для прямого скачка, записанными по нормальной составляющей числа Маха $M_{1n} = M_1 \sin\theta_c = 1{,}38$: значения p_2 и ρ_2 совпали.

3.2 Задание 2

Требуется найти корень уравнения $x^4 - 3x^3 + 5x - 10 = 0$ на отрезке $[-5; 0]$ методом Ньютона. Аналитические производные:

$$f'(x)=4x^3-9x^2+5=(x-1)(4x^2-5x-5),\ f''(x)=12x^2-18x. \qquad (16)$$

Отделение корня. $f(-5)=965>0$, $f(0)=-10,0<0$, т. е. корень на отрезке существует. Однако производная обращается в нуль в точке $x=\frac{5-\sqrt{105}}{8}\approx -0,656$, лежащей внутри отрезка (рисунок 3.3), поэтому условия знакопостоянства производных на всём $[-5;0]$ не выполняются. Так как $f(-0,656)=-12,2<0$, на отрезке $[-0,656;0]$ функция отрицательна, и корень лежит левее. По графику он находится между -2 и -1.

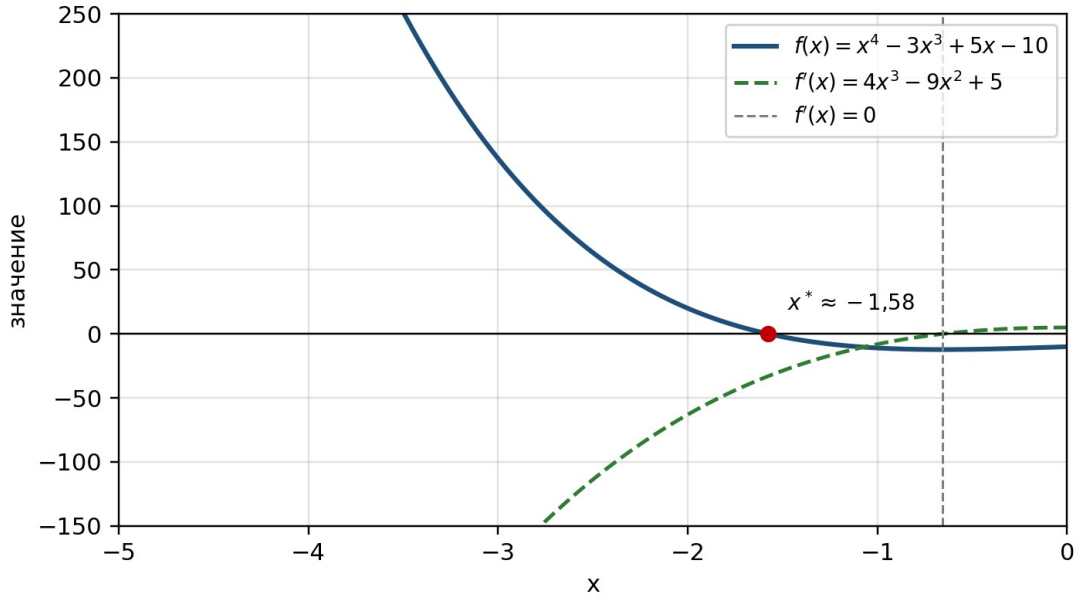


Рисунок 3.3 – График функции $f(x)=x^4-3x^3+5x-10$ и её производной на отрезке $[-5;0]$

На отрезке $[-2;-1]$: $f(-2)=20,0$, $f(-1)=-11,0$, $-63,0\le f'(x)\le -8,00$, $30,0\le f''(x)\le 84,0$ — корень единственный. Но условие сходимости метода Ньютона (3) на нём нарушается: при $x=-1$ имеем $|ff''|=330>(f')^2=64,0$, а максимум отношения $|ff''|/(f')^2$ на отрезке равен $5,16$. Поэтому отрезок сужен до $[-2;-1,5]$, так как $f(-1,5)=-2,31<0$ (рисунок 3.4).

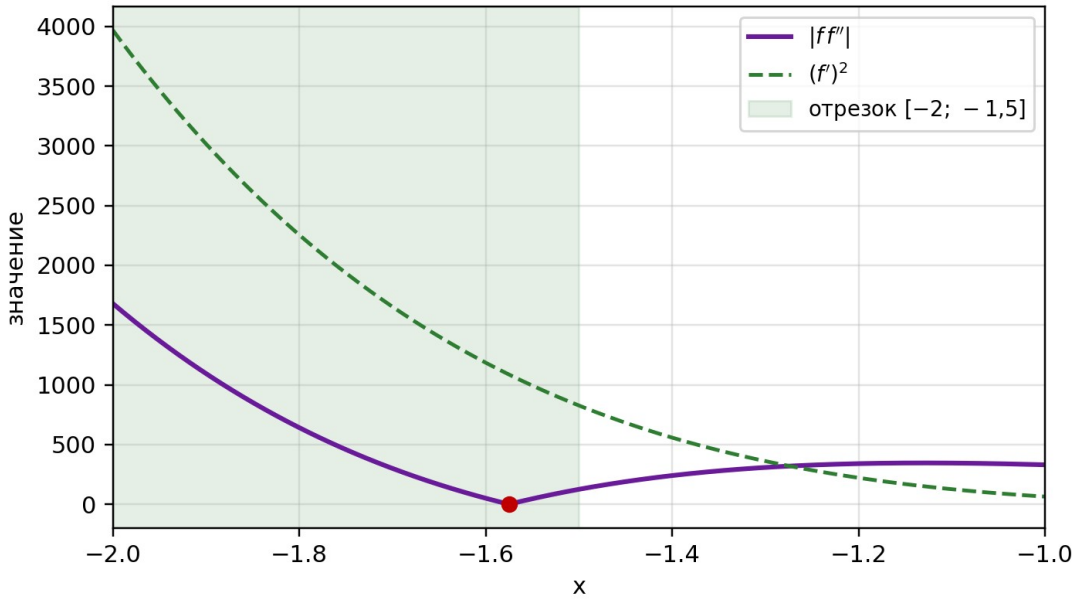


Рисунок 3.4 – Проверка условия сходимости метода Ньютона $|ff''|<(f')^2$

Проверка на отрезке $[-2;-1{,}5]$ (функции `check_root_conditions` и `check_newton_condition`, приложение А):

1. $f(-2)=20{,}0>0$, $f(-1{,}5)=-2{,}31<0$ — корень существует.
2. $-63{,}0\le f'(x)\le -28{,}8$, т. е. $f'<0$ — корень единственный.
3. $54{,}0\le f''(x)\le 84{,}0$, т. е. $f''>0$ — вторая производная сохраняет знак.
4. $max|ff''|/(f')^2=0{,}423<1$ — условие (3) выполнено на всём отрезке.

Начальное приближение выбрано на конце отрезка, где $f(x_0)f''(x_0)>0$: $x_0=-2$. Итерации по формуле (2) приведены в таблице 3.3.

Таблица 3.3 – Итерации метода Ньютона в задании 2

n	x_n	$f(x_n)$	$\varepsilon_n=\lvert x_n-x^{\ast}\rvert$	$\varepsilon_n/\varepsilon_{n-1}^2$
0	$-2{,}00$	$20{,}0$	$0{,}425$	—
1	$-1{,}68$	$3{,}89$	$0{,}108$	$0{,}595$
2	$-1{,}58$	$0{,}302$	$0{,}00909$	$0{,}787$
3	$-1{,}58$	$0{,}00238$	$7{,}22\cdot10^{-5}$	$0{,}873$
4	$-1{,}58$	$1{,}51\cdot10^{-7}$	$4{,}59\cdot10^{-9}$	$0{,}882$
5	$-1{,}58$	0	0	—

Корень найден за 5 итераций: $x^{\ast}=-1{,}58$. Проверка с помощью функции `scipy.optimize.brentq` дала то же значение в пределах заданной точности.

3.3 Задание 3

Требуется применить метод Ньютона (метод задания 2) для решения уравнения (8) с исходными данными задания 1 и сравнить результаты с методом секущих.

Проверка перед применением метода. Используется тот же отрезок $[20°, 40°]$. Условия следствия теоремы Больцано–Коши для него проверены в задании 1: корень существует и единственный, f' и f'' сохраняют знак. Дополнительно проверено условие сходимости метода Ньютона (3): максимум отношения $|ff''|/(f')^2$ на отрезке равен $0{,}521<1$, т. е. условие выполнено на всём отрезке (рисунок 3.5).

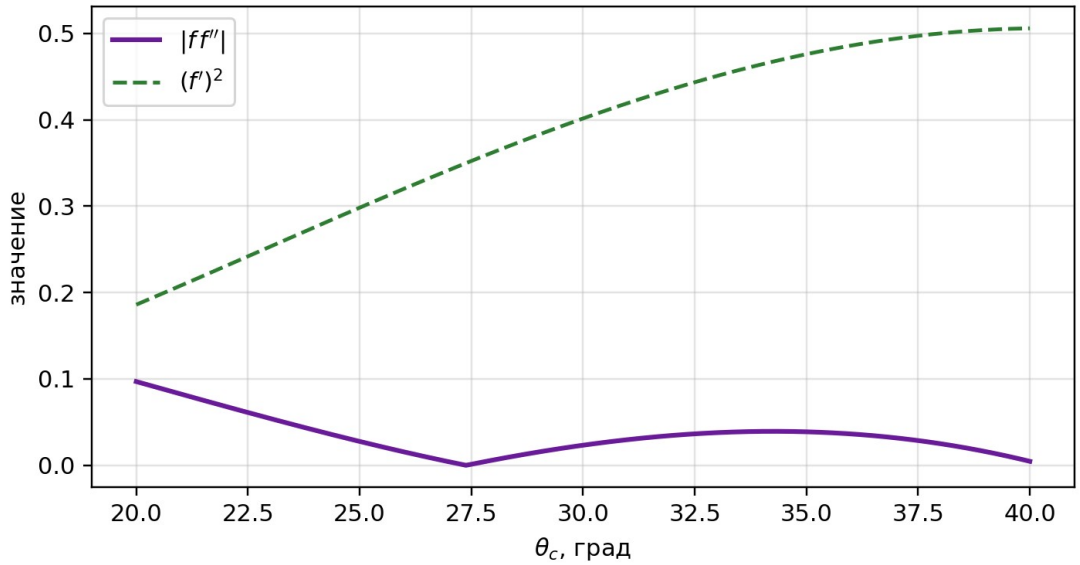


Рисунок 3.5 – Проверка условия сходимости метода Ньютона для уравнения (8) на отрезке $[20°, 40°]$

Так как $f(a)f''(a) = -0{,}0969 < 0$, а $f(b)f''(b) = 0{,}00462 > 0$, начальное приближение взято на правой границе: $\theta_0 = 40°$. Производные вычисляются по аналитическим формулам (12)–(13). Итерации приведены в таблице 3.4.

Таблица 3.4 – Итерации метода Ньютона в задании 3

n	θ_n, °	$f(\theta_n)$	$\varepsilon_n = \lvert\theta_n - \theta^{*}\rvert$, **рад**	$\varepsilon_n / \varepsilon_{n-1}^2$
0	40,0	0,147	0,220	—
1	28,1	0,00775	0,0130	0,267
2	27,4	$8{,}04 \cdot 10^{-5}$	$1{,}36 \cdot 10^{-4}$	0,810
3	27,4	$9{,}36 \cdot 10^{-9}$	$1{,}58 \cdot 10^{-8}$	0,856
4	27,4	$1{,}67 \cdot 10^{-16}$	$3{,}33 \cdot 10^{-16}$	1,33

Отношение $\varepsilon_i / \varepsilon_{i-1}^2$ вблизи корня остаётся ограниченным, что подтверждает второй порядок сходимости метода Ньютона (4).

Сравнение методов. Для сравнения времени выбрано **реальное (астрономическое) время** выполнения, измеренное функцией `time.perf_counter_ns`. Так как одно решение занимает единицы микросекунд, что сравнимо с разрешением таймера, время усреднено по 10 000 повторным запускам каждого метода (функция `measure_time`, приложение А). В замер включены только итерации методов, без проверки условий и расчёта параметров потока. Результаты сравнения приведены в таблице 3.5, сходимость обоих методов показана на рисунке 3.6.

Таблица 3.5 – Сравнение методов решения уравнения (8)

Параметр	**Метод секущих (задание 1)**	**Метод Ньютона (задание 3)**
Угол наклона КСУ θ_c, °	27,4	27,4
Давление p_2, Па	$1{,}54 \cdot 10^5$	$1{,}54 \cdot 10^5$
Температура T_2, К	358	358
Плотность ρ_2, кг/м³	1,50	1,50
Число Маха M_2	2,51	2,51
Режим течения	сверхзвуковой	сверхзвуковой
Число итераций	5	4
Число вычислений f / f'	7 / 0	4 / 4
Реальное время расчёта, мкс	6,23	6,08
Начальные данные	$x_0 = 20°$, $x_1 = 40°$	$\theta_0 = 40°$
Требуемые проверки	Больцано–Коши	Больцано–Коши и условие (3)

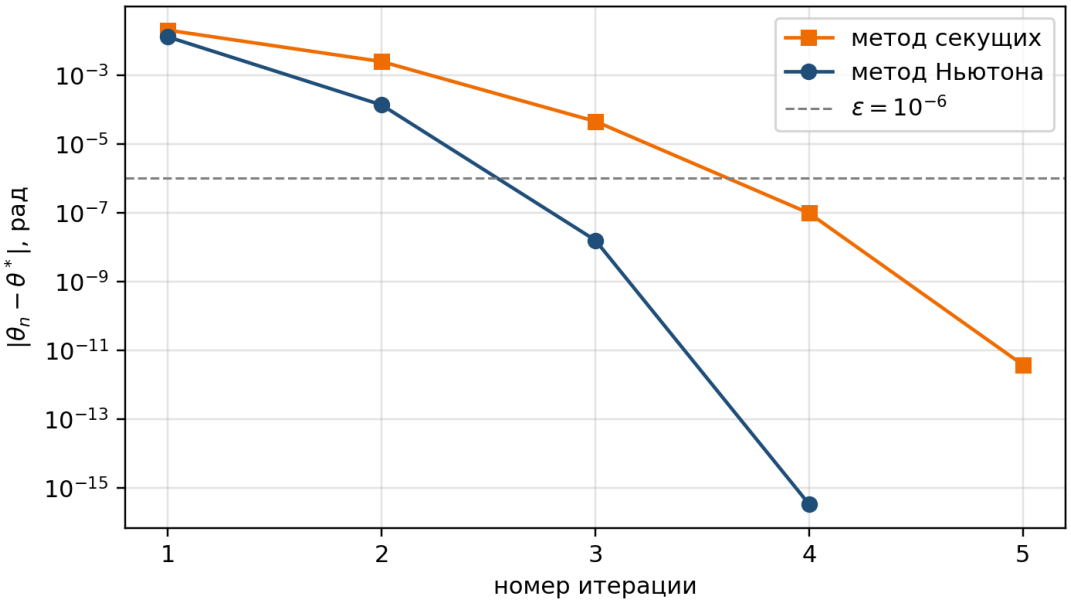


Рисунок 3.6 – Ошибка приближения в зависимости от номера итерации

Значения угла, полученные двумя методами, совпадают (разница $2{,}2 \cdot 10^{-10}$°, что много меньше заданной точности), поэтому совпадают и все параметры за скачком. Метод Ньютона сошёлся за 4 итерации против 5 у метода секущих: у метода Ньютона число верных знаков примерно удваивается на каждой итерации, у метода секущих ошибка убывает медленнее (рисунок 3.6). При этом по реальному времени методы практически равны (6,08 и 6,23 мкс): итерация метода Ньютона дороже, так как на каждой из них вычисляются f и f', тогда как метод секущих вычисляет только одно новое значение f, запоминая предыдущее. В сумме метод секущих выполнил 7 вычислений функции, метод Ньютона — 8 вычислений f и f'.

4 Вывод

В ходе лабораторной работы изучены условно сходящиеся итерационные методы решения нелинейных уравнений — метод Ньютона и метод секущих — и решены три задания варианта 9.

В задании 1 методом секущих найден угол наклона косого скачка уплотнения на клине с углом $\beta = 10°$ при $M_1 = 3$: $\theta_c = 27{,}4°$. Из двух корней уравнения (8) на $(0, \pi/2)$ выбран меньший (слабый скачок), так как больший соответствует неустойчивому решению. Перед применением метода на отрезке $[20°, 40°]$ проверены существование и единственность корня и знакопостоянство производных. Рассчитаны параметры за скачком: $p_2 = 1{,}54 \cdot 10^5$ Па, $T_2 = 358$ К, $\rho_2 = 1{,}50$ кг/м³, $M_2 = 2{,}51$. Давление, температура и плотность за КСУ растут, число Маха падает, но остаётся больше единицы — течение после скачка сверхзвуковое.

В задании 2 методом Ньютона найден корень уравнения $x^4 - 3x^3 + 5x - 10 = 0$ на $[-5; 0]$: $x^* = -1{,}58$ за 5 итераций. Исходный отрезок пришлось сузить до $[-2; -1{,}5]$: на $[-5; 0]$ производная меняет знак, а на $[-2; -1]$ нарушается условие сходимости $|f f''| < (f')^2$.

В задании 3 метод Ньютона применён к уравнению (8). Оба метода дали одинаковый результат $\theta_c = 27{,}4°$, однако метод Ньютона сошёлся за 4 итерации против 5 у метода секущих. По реальному времени методы оказались практически равны ($6{,}08$ и $6{,}23$ мкс), так как итерация метода Ньютона требует вычисления производной. Метод Ньютона выгоден, когда производная вычисляется просто и нужна высокая точность за малое число итераций, но требует аналитических производных и проверки условия сходимости на всём отрезке. Метод секущих не требует производных и может стартовать с границ отрезка, на котором выполнены условия теоремы Больцано–Коши, ценой несколько большего числа итераций.

Список использованных источников

1. Определение параметров за косым скачком уплотнения на клине: методические указания к лабораторной работе № 1. — М.: МАИ, 2026.
2. Лукьяненко Д. В. Численные методы. — М.: МГУ им. М. В. Ломоносова. — 101 с.
3. Савчук В. Ф. Методы численного анализа. — Брест: БрГУ, 2013. — 403 с.
4. Калугин В. Т. Моделирование процессов обтекания и управления аэродинамическими характеристиками летательных аппаратов. — М.: МГТУ им. Н. Э. Баумана, 2011. — 528 с.

Приложение А. Программный код

Ниже приведён код ноутбука `matmod.ipynb`: реализация методов секущих и Ньютона, проверок условий существования корня и сходимости, расчёта параметров за КСУ, построения графиков и вызовов для заданий 1–3. Ячейки разделены комментариями `# %%`.

```
# %% Ячейка 1
import numpy as np
import pandas as pd
import math
import time
import matplotlib.pyplot as plt
from IPython.display import display, Math

eps = 1e-6
max_iterations = 100

sig = lambda value: np.format_float_positional(value, precision=3,
unique=False, fractional=False, trim='k').rstrip('.')

# %% Ячейка 2
def secant(f, x0, x1, eps=eps, max_iterations=max_iterations):
    f0, f1 = f(x0), f(x1)
    iterations = 0
    condition = True

    while condition and iterations < max_iterations:
        iterations += 1
        x_i = x1 - (x1 - x0) / (f1 - f0) * f1
        condition = abs(x_i - x1) >= eps
        x0, f0 = x1, f1
        x1, f1 = x_i, f(x_i)

    return x1, iterations

# %% Ячейка 3
def newton(f, derivative, x0, eps=eps, max_iterations=max_iterations):
    x_i = x0
    iterations = 0
    condition = True

    while condition and iterations < max_iterations:
        iterations += 1
        delta = f(x_i) / derivative(x_i)
        x_i -= delta
        condition = abs(delta) >= eps

    return x_i, iterations



def newton_start(f, second_derivative, a, b):
    return a if f(a) * second_derivative(a) > 0 else b
```

```
# %% Ячейка 4
def is_constant_sign(values):
    return bool(np.all(values > 0) or np.all(values < 0))



def check_root_conditions(f, derivative, second_derivative, a, b,
points=10_000):
    x = np.linspace(a, b, points)
    return {
        "f(a) f(b) < 0": bool(f(a) * f(b) < 0),
        "f' не меняет знак": is_constant_sign(derivative(x)),
        "f'' не меняет знак": is_constant_sign(second_derivative(x)),
    }



def check_newton_condition(f, derivative, second_derivative, a, b,
points=10_000):
    x = np.linspace(a, b, points)
    return bool(np.all(np.abs(f(x) * second_derivative(x)) < derivative(x) **
2))

# %% Ячейка 5
def measure_time(method, *args, repeats=10_000):
    start = time.perf_counter_ns()
    for _ in range(repeats):
        method(*args)
    return (time.perf_counter_ns() - start) / repeats / 1000

# %% Ячейка 6
p1 = 75000
T1 = 288
M1 = 3
beta = math.radians(10)

kappa = 1.4
R = 287

display(pd.DataFrame(
    {
        "Параметр": ["p1", "T1", "M1", "β", "κ", "R"],
        "Значение": [p1, T1, M1, 10, kappa, R],
        "Размерность": ["Па", "К", "—", "°", "—", "Дж/(кг·К)"],
    }
))

# %% Ячейка 7
rho1 = p1 / (R * T1)
a1 = math.sqrt(kappa * R * T1)
v1 = M1 * a1

display(Math(fr"\rho_1 = {sig(rho1)}\ кг/м^3, \quad a_1 = {sig(a1)}\ м/с,
\quad v_1 = {sig(v1)}\ м/с"))

# %% Ячейка 8
k = (kappa + 1) / 2

shock_equation = lambda t: np.sin(t) ** 2 - k * np.sin(t) * np.sin(beta) /
np.cos(t - beta) - 1 / M1 ** 2
```

```
# %% Ячейка 9
theta = np.linspace(1e-3, np.pi / 2, 500)

plt.axhline(0, color='lightgreen', linewidth=2)
plt.plot(np.degrees(theta), shock_equation(theta))
plt.xlabel(r'$\theta_c$, °')
plt.ylabel(r'$f(\theta_c)$')
plt.grid(True)
plt.show()

# %% Ячейка 10
shock_derivative = lambda t: np.sin(2 * t) - k * np.sin(beta) *
np.cos(beta) / np.cos(t - beta) ** 2
shock_second_derivative = lambda t: (
    2 * np.cos(2 * t) - 2 * k * np.sin(beta) * np.cos(beta) * np.sin(t -
beta) / np.cos(t - beta) ** 3
)

theta_a, theta_b = math.radians(20), math.radians(40)

# %% Ячейка 11
theta = np.linspace(theta_a, theta_b, 200)

plt.axhline(0, color='lightgreen', linewidth=2)
plt.plot(np.degrees(theta), shock_equation(theta), label=r"$f$")
plt.plot(np.degrees(theta), shock_derivative(theta), label=r"$f'$")
plt.plot(np.degrees(theta), shock_second_derivative(theta), label=r"$f''$")
plt.xlabel(r'$\theta_c$, °')
plt.legend()
plt.grid(True)
plt.show()

# %% Ячейка 12
check_root_conditions(shock_equation, shock_derivative,
shock_second_derivative, theta_a, theta_b)

# %% Ячейка 13
theta_secant, iterations_secant = secant(shock_equation, theta_a, theta_b)

display(Math(fr"\theta_c = {sig(math.degrees(theta_secant))}^\circ"))
print(f"Итераций: {iterations_secant}")

# %% Ячейка 14
def shock_parameters(theta_c):
    v1n = v1 * math.sin(theta_c)
    v1t = v1 * math.cos(theta_c)

    u1 = 1 - math.sin(theta_c - beta) * math.cos(theta_c) /
(math.sin(theta_c) * math.cos(theta_c - beta))

    rho2 = rho1 / (1 - u1)
    p2 = p1 + rho1 * u1 * v1n ** 2
    T2 = p2 / (rho2 * R)

    v2n = v1n * (1 - u1)
    v2 = math.sqrt(v2n ** 2 + v1t ** 2)
    a2 = math.sqrt(kappa * R * T2)
```

```
    M2 = v2 / a2

    return {"p2": p2, "T2": T2, "rho2": rho2, "v2": v2, "a2": a2, "M2": M2}

params = shock_parameters(theta_secant)

# %% Ячейка 15
results_1 = pd.DataFrame(
    {
        "Параметр": ["θc", "p2", "T2", "ρ2", "v2", "a2", "M2"],
        "Значение": [sig(math.degrees(theta_secant))] + [sig(params[key]) for
key in ["p2", "T2", "rho2", "v2", "a2", "M2"]],
        "Размерность": ["°", "Па", "K", "кг/м³", "м/с", "м/с", "—"],
    }
)
results_1

# %% Ячейка 16
regime = "сверхзвуковое" if params["M2"] > 1 else "дозвуковое"
print(f"M2 = {sig(params['M2'])} -> течение за КСУ {regime}")

# %% Ячейка 17
equation = lambda x: x ** 4 - 3 * x ** 3 + 5 * x - 10
derivative = lambda x: 4 * x ** 3 - 9 * x ** 2 + 5
second_derivative = lambda x: 12 * x ** 2 - 18 * x

a, b = -5, 0

# %% Ячейка 18
x = np.linspace(a, b, 200)

plt.axhline(0, color='lightgreen', linewidth=2)
plt.plot(x, equation(x), label=r"$f$")
plt.plot(x, derivative(x), label=r"$f'$")
plt.legend()
plt.grid(True)
plt.show()

# %% Ячейка 19
check_root_conditions(equation, derivative, second_derivative, a, b)

# %% Ячейка 20
a, b = -2, -1

check_root_conditions(equation, derivative, second_derivative, a, b),
check_newton_condition(equation, derivative, second_derivative, a, b)

# %% Ячейка 21
a, b = -2, -1.5

check_root_conditions(equation, derivative, second_derivative, a, b),
check_newton_condition(equation, derivative, second_derivative, a, b)

# %% Ячейка 22
x = np.linspace(a, b, 200)

plt.axhline(0, color='lightgreen', linewidth=2)
```

```
plt.plot(x, equation(x), label=r"$f$")
plt.plot(x, np.abs(equation(x) * second_derivative(x)), label=r"$|f f''|$")
plt.plot(x, derivative(x) ** 2, label=r"$(f')^2$")
plt.legend()
plt.grid(True)
plt.show()


# %% Ячейка 23
x0 = newton_start(equation, second_derivative, a, b)
x_newton, iterations = newton(equation, derivative, x0)

display(Math(fr"x_0 = {x0}, \quad x^* = {sig(x_newton)}"))
print(f"Итераций: {iterations}")


# %% Ячейка 24
check_newton_condition(shock_equation, shock_derivative,
shock_second_derivative, theta_a, theta_b)

# %% Ячейка 25
theta = np.linspace(theta_a, theta_b, 200)

plt.plot(np.degrees(theta), np.abs(shock_equation(theta) *
shock_second_derivative(theta)), label=r"$|f f''|$")
plt.plot(np.degrees(theta), shock_derivative(theta) ** 2, label=r"$(f')^2$")
plt.xlabel(r'$\theta_c$, °')
plt.legend()
plt.grid(True)
plt.show()

# %% Ячейка 26
theta0 = newton_start(shock_equation, shock_second_derivative, theta_a,
theta_b)
theta_newton, iterations_newton = newton(shock_equation, shock_derivative,
theta0)

display(Math(fr"\theta_0 = {sig(math.degrees(theta0))}^\circ, \quad \theta_c
= {sig(math.degrees(theta_newton))}^\circ"))
print(f"Итераций: {iterations_newton}")

# %% Ячейка 27
time_secant = measure_time(secant, shock_equation, theta_a, theta_b)
time_newton = measure_time(newton, shock_equation, shock_derivative, theta0)

params_newton = shock_parameters(theta_newton)

comparison = pd.DataFrame(
    {
        "θc, °": [sig(math.degrees(theta_secant)),
sig(math.degrees(theta_newton))],
        "p2, Па": [sig(params["p2"]), sig(params_newton["p2"])],
        "T2, К": [sig(params["T2"]), sig(params_newton["T2"])],
        "ρ2, кг/м³": [sig(params["rho2"]), sig(params_newton["rho2"])],
        "M2": [sig(params["M2"]), sig(params_newton["M2"])],
        "Итерации": [iterations_secant, iterations_newton],
        "Реальное время, мкс": [sig(time_secant), sig(time_newton)],
    },
    index=["Метод секущих", "Метод Ньютона"],
)
comparison
```

```
# %% Ячейка 28
display(Math(fr"|\theta_{{MC}} - \theta_{{MH}}| =
{abs(math.degrees(theta_secant - theta_newton)):.2e}^\circ"))

# %% Ячейка 29
from scipy.optimize import brentq

theta_root = brentq(shock_equation, theta_a, theta_b)
root = brentq(equation, a, b)

# %% Ячейка 30
np.allclose(theta_secant, theta_root, atol=eps), np.allclose(theta_newton,
theta_root, atol=eps)

# %% Ячейка 31
np.allclose(x_newton, root, atol=eps)

# %% Ячейка 32
M1n = M1 * math.sin(theta_secant)
p2_check = p1 * (1 + 2 * kappa / (kappa + 1) * (M1n ** 2 - 1))
rho2_check = rho1 * (kappa + 1) * M1n ** 2 / ((kappa - 1) * M1n ** 2 + 2)

np.allclose([params["p2"], params["rho2"]], [p2_check, rho2_check])
```